

Atama

CS 30700 - Spring 2026

Manya Gupta, Emma Jiang, Pawani Surthi, Lizzy Zhou

Team 20

Index

• Purpose	2
○ Functional Requirements	
○ Non-Functional Requirements	
• Design Outline	6
○ Client-Server Model	
○ High-Level Overview	
○ High-Level Structure Diagram	
• Design Issues	8
○ Functional Issues	
○ Non-Functional Issues	
• Design Detail	12
○ Class Design	
○ Class Interactions	
○ Database Design	
○ Sequence of Events Diagrams	
○ Activity Diagram	
○ API routes	

Purpose

Students have to study—it's what it means to be a student. While going to class and completing homework is usually easygoing, come exam season, it's time to lock in. Students frantically search for class materials, scrounge for practice exams from semesters past, and throw together flashcard sets that they review only five minutes before the exam. Sometimes it seems that what's more stressful than the studying itself is *finding* the material to study with.

In times like these, students often turn to applications such as Quizlet, Anki, Studocu, and RemNote. However, these applications have differing and scattered features, and lack standardization and organization. Most glaringly, they are often blocked or limited behind ads and paywalls, putting some students at a disadvantage. Our study application is centralized, with quick study material generation, easy consolidation of study materials between students, and encouraged student interaction and collaboration. With tools like flashcards, study guides, and practice tests supplemented with generative AI, as well as a shared resource drive and study groups, students can now do all of their studying in one streamlined app: Atama.

Functional Requirements

1. Users can manage their account and settings

As a user, I would like to:

1. register for an account with an email address.
2. set a unique account username.
3. log into my account.
4. manage my account information such as username, password, privacy settings, and profile picture.
5. toggle dark/light mode.
6. view my library of study materials I created and allow others to view my library if I set it to public.
7. opt out of or into AI-generated study content.

2. Users can create and use study materials

As a user, I would like to:

1. create flashcard sets.
2. study flashcard sets.

3. create different types of flashcards, such as drag-and-drop labels onto a diagram, putting steps in order, and fill-in-the-blank.
4. access and use other users' flashcard sets to study.
5. rate others' flashcard sets with a 5-star system.
6. review others' flashcard sets with a tagging system with tags such as "Quick to study", "All class content", "Professor-specific", etc.
7. externally share flashcard sets and other study materials through a generated shareable link.
8. upload my notes or slideshows in PDF form so that the application generates flashcards for me, using AI. Otherwise, formatted text can also be used to create flashcards.
9. have the application generate practice tests from my flashcard set(s) or other study materials, using AI if enabled. Otherwise, formatted text can also be used to create practice tests.
10. download or export my flashcard sets and practice tests.
11. create a concept map using my flashcards through drag-and-drop.
12. to have the application generate a concept map for me using generative AI.
13. choose between different studying tools, like flashcards and practice tests.
14. study using individual games such as memory.
15. study using collaborative games structured like Kahoot.

3. Users can join a university, courses, and study groups

As a user, I would like to:

1. search for and join my school.
2. join specific classes in my school for the semester.
3. upload my schedule and automatically join specific classes in my school for the semester.
4. automatically leave classes once the semester is over.
5. have student verification for joining a university
6. search for student content for specific classes within my school.
7. upload, access, and download study materials in the shared drive for my specific class.
8. filter searches in the shared drives to find content more easily.
9. join study groups for my specific classes.

10. participate in and observe leaderboards regarding how much time members of my study group spend practicing, or other statistics.
11. send nudges (via notifications/emails) to members in my study group to encourage them to study or practice.

4. Users have access to other helpful studying features

As a user, I would like to:

1. set study schedules or goals so that I know when and how much to practice.
2. receive desktop notifications that tell me when to start studying based on my set study schedule or goals.
3. observe and maintain a study streak.
4. set an exam date and have a countdown as a reminder.
5. view the progress I have made in studying a set and how much time I have spent on it.
6. utilize study time management methods with a built in timer, such as Pomodoro.
7. report content that is inappropriate for the purpose of the application.
8. migrate study materials from other platforms (e.g., Quizlet flashcards) to this platform.

Non-Functional Requirements

1. Client Requirements

As a client, I would like:

1. to use the application online as a web application.

2. Server Requirements

As a developer, I would like:

1. for the server to store use and system data in a database.
2. for the server to be able to handle at least 5,000 users at a time.
3. for the server to be able to retrieve data from the database in accordance to the user's actions.
4. for the server to be able to handle front-end requests.

3. Design Requirements

As a developer, I would like:

1. for the application to be aesthetically pleasing and easy to use.

2. for the application to be organized and visually structured for ease of finding study materials.

4. Performance Requirements

As a developer, I would like:

1. a download speed of 50-100 Mbps from a university class's shared drive.
2. an upload speed of 2-10 Mbps to a university class's shared drive.
3. for a shared drive to hold up to 15 Gb of storage.
4. for a user's personal study material library to hold up to 2 Gb of storage.
5. the search bar to have a response time of 800 ms or less.

5. Security Requirements

As a developer, I would like:

1. accurate verification of student university emails in order for a user to join a university.
2. adequate moderation of created or uploaded content to ensure safe and appropriate behavior on the application.
3. for the server to be secure such that non-authorized requests are denied.
4. for user data to be anonymized or secured on the user's device before being transmitted or handled by our system.
5. for users to have the option to toggle generative AI features on or off in their settings.

Design Outline

This project will be a web application that allows users to create and study flashcards and other study materials, find other study materials from specific classes, and connect with other students. This application will follow the client-server model where a single server will handle requests from multiple clients simultaneously using the SpringBoot framework in Java.

Client-Server Model

1. Web client

- a. The web client, running in each user's browser, will be the interface to our system.
- b. The web client will retrieve and send data to our web server via HTTP requests, passing data in JSON format.
- c. The data retrieved from the server will be parsed, formatted, and presented to the user to display information and render changes in the user interface.

2. API server

- a. The server will receive and handle HTTP requests from clients.
- b. The server will validate requests and send queries to the database to add, modify, or retrieve data on demand.
- c. The server will generate appropriate responses and send them to targeted clients.

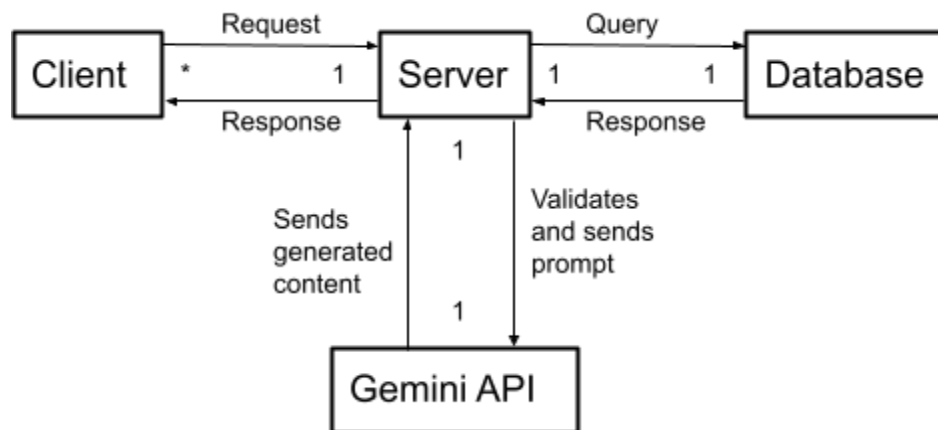
3. Database

- a. A relational database will store all of the data used in the application, such as user information, flashcard sets, and schools and courses.
- b. The database will respond to queries from the server and send the requested data back.

High-Level Overview

This web application follows a client-server architecture, with a React frontend communicating with a Spring Boot backend using RESTful APIs using JSON over HTTP. The backend is designed to support concurrent access from many clients, using multithreaded request handling to ensure scalability. A relational database is used for persistent data storage, while application logic, authentication, and validation are handled by the server. The system also integrates with an LLM API to generate flashcards and study content automatically from user-provided notes.

High-Level Structure Diagram



Design Issues

Functional Issues

1. Do users need to log in to use the application?

- Option 1: Yes, or else no access to the application
- Option 2: No, but they are given limited access to the application
- Option 3: No, they can have complete access to the application

Choice: Option 2

Justification: While we do not want to paywall or restrict parts of our application, we still want users to sign up and create accounts to achieve the full utility of Atama. This is due to multiple reasons. The first reason being we want content that users create to be tied to an account. Secondly, users that are not logged in should not be able to upload or alter shared drive materials. If they are not logged in, users will have limited access to the application. They may access certain features such as public flashcard sets, but not features such as different study modes, university shared drives, etc. Once the user creates an account or logs in, they will be granted full user privileges.

2. How should the shared drive contents be displayed?

- Option 1: All contents listed in order of a particular attribute, which can be changed and chosen from options (e.g., upload date, alphabetical, semester of year, etc.)
- Option 2: All contents listed in order of semester/year, but visually delineated based on study material type (e.g., flashcard sets, study guides, exam reviews, past exams)

Choice: Option 2

Justification: We believe Option 2 would be most beneficial as we do not intend to provide users with a search bar to browse the contents of the drive. Therefore, for a user, it'll be easiest to find material by the relevance of the current/past semester which a document would be given by the user uploading it. Furthermore, we don't believe users will benefit from sorting by categories like upload date outlined by Option 1, as we imagine that when users use the shared drive, they access it with a purpose of retrieving a certain type and version of study material for the semester they are currently in.

3. How should we verify that users are students enrolled at a university?

- Option 1: No verification
- Option 2: Verification through school email
- Option 3: Verification through other means, such as SheerID

Choice: Option 2

Justification: We chose to verify students through their school email by checking their email domain (e.g., @purdue.edu). Option 1 isn't well-suited or secure, as this would allow any user to upload whatever materials to the shared drive. To be able to ensure the quality and relevance of materials, users should be from the organization. While Option 3 is more secure, SheerID is not within the budget or scope of the task of verification. Option 2 is best as although there are ways to circumvent this check, we believe those methods are far too uncommon to warrant a stricter verification process outlined by Option 3.

4. How should we let users rate and review flashcard sets?

- Option 1: 5-star rating system
- Option 2: Tag review system, similar to that of RateMyProfessors
- Option 3: Written review system with word limit
- Option 4: Combination of the above options

Choice: Option 4

Justification: We decided to go with a star rating and tag review system combination. While the written review system would be informational for users, there would be a greater need for content and spam moderation. Furthermore, it would place a greater strain on our system storage capacities as storing string requires more space. With the star rating system, users can quickly leave numerical feedback on a flashcard set. This provides a good overview of the flashcard set quality for users who are searching for sets to study.

With the tag review system, users can mark flashcard sets with predefined tags containing short descriptions, such as "Quick to study", "All class content covered", "Professor-specific material", etc. This is descriptive enough to convey information about the flashcard set, without total freedom that would require content moderation. This system will also aid us when we construct the search bar.

5. What users should be able to access and upload to the shared drive of a university class?

- Option 1: Only verified members of the university who are enrolled in the class in real life
- Option 2: Only verified members of the university
- Option 3: Any logged in user
- Option 4: Any user

Choice: Option 2

Justification: While Option 1 is the most secure, we have no way of verifying if a user is enrolled in a certain course in real life without involving the university's input which is out of the scope for this project. We believe Option 2 provides sufficient security to shared drives as it allows verified members of a university privileges, reducing the amount of potential bad actors. While this option requires a level of good faith to be given to the verified users, we believe that if bad actors present themselves as an issue, they can be fixed by the restore functionality that comes with some Postgres databases. Options 3 and 4, we believe, allow for too many users to have direct access to materials they do not need. We apply the security principle that access should be strict to begin with.

Non-Functional Issues

1. What type of database is most appropriate for our data?

- Option 1: A SQL database like Postgres
- Option 2: A NoSQL database like MongoDB
- Option 3: Simply data structures stored at the server level

Choice: Option 1

Justification: A SQL database like Postgres is most appropriate because it benefits from both the relation model that is inherited to SQL databases and the ACID properties. Our core features, including user accounts and flashcard sets, require a consistent structure that can be easily represented through schemas. Furthermore, the foreign key structure and JOIN will allow easy linking between objects like a user to their flashcard sets. Lastly, the ACID properties will be immensely useful to ensure data corruption or loss during server-database interactions. In respect to option 2, Atama's data isn't unstructured enough to warrant the loss of relational integrity and storing data at the server level isn't scalable.

2. What backend language/framework should we use?

- Option 1: Python + FastAPI
- Option 2: Java + SpringBoot
- Option 3: Node.js + Express

Choice: Option 2

Justification: We picked Java with SpringBoot to ensure a secure, scalable, and maintainable foundation for our backend. We picked Java because it is a statically typed language; this will help us catch errors at compile time rather than runtime, which will reduce bugs in production. Furthermore, SpringBoot's modular architecture allows us to build a modular system that will make it easier to separate the logic for flashcards, the AI-powered study guide generator, etc. Also, the SpringBoot data JPA integrates easily with Postgres, and allows mapping Java objects to database tables. This will make CRUD operations easier. While FastAPI is fast to develop with, option one isn't the best as python is a dynamically typed language which means the risk of runtime issues is too high compared to Java's stricter ecosystem rules. Option three wasn't chosen because Express doesn't provide a standard structure for database management (among others) which would mean we would have to spend time deciding and configuring third-party libraries.

3. What frontend language/framework should we use?

- Option 1: React
- Option 2: Angular
- Option 3: Swift

Choice: Option 1

Justification: React has component-based architecture, which allows us to build reusable UI elements. This will help speed up development and ensure consistency across elements that will make our product more cohesive. Furthermore, our product requires frequent UI updates (switching flashcards), so React's Virtual DOM will ensure that only necessary parts are rendered. This provides a better overall user experience. Lastly, React has a large community, and with our members' low experience with frontend work, the wealth of documentation and pre-built components will aid in development.

Design Details

Class Design

User

Represents a person using the application who is signed into their account. Contains hidden identifying information related to sign-up, as well as public data viewable by other Users.

- ID (Integer): The unique user ID assigned to the user upon account creation.
- Username (String): The unique username of the user. Can be auto-generated by the email provided during sign-up or chosen by the user (provided it does not already belong to another User). Publicly available to other users. Capped at 15 characters.
- Password (String): The password hash for the User's account. Not publicly available to other users.
- Email (String): The email associated with the account. Required to create an account. Not publicly available.
- University (University): An optional identifier that contains the university that the user attends. Publicly visible if the User opts to make it public.
 - University Publicness (Boolean): Whether or not the User has opted to make their University information public.
- Profile Photo (Image): An image provided by the user for their profile. If no photo is provided, then this holds a default profile photo. Publicly available.
- Goals (Int): An optional goal of minutes of studying per week the user has set for themselves. Not publicly available.
- Streak (Int): The number of consecutive days that a User has completed at least one study activity. Publicly visible if the User opts to make it public.
 - Streak Publicness (Boolean): Whether or not the User has opted to make their streak public.

Study Group

Represents a group of users that have voluntarily joined a study group in the application. Must be attending the same University. Can be created by a student that is verified at a University.

- ID (Integer): The unique ID of the Study Group.

- Name (String): A short title for the study group, inputted by the creator. For example, "CS 182 FA 25". Capped at 60 characters.
- Description (String): An optional description of the study group, inputted by the creator. Capped at 150 characters.
- University (University): The University that the study group is associated with.
- Members (List[Users]): A list of users that are in the study group.
- Latest Activity Date (Time): The date the Study Group was last active.

Flashcard : Interface

Represents a single flashcard. Must be part of a Flashcard Set. The interface is extended to support different types of flashcards.

- ID (Integer): The unique ID of the Flashcard.

Normal Flashcard: extends Flashcard

- Term (String): The term of the flashcard.
- Definition (String): The definition of the flashcard.

Drag and Drop Flashcard: extends Flashcard

- Image (Image): The image uploaded by the user.
- Draggable Labels (List[String]): Label for the image.
- Correct Mapping (List[String]): Used to verify the correct answer.

Fill in the Blank Flashcard: extends Flashcard

- Sentence Template (String): The sentence to be filled in.
- Answer (String): The term to fill the sentence correctly.

Steps Flashcard: extends Flashcard

- Title (String): Title to label the steps. For example, "Steps of the Krebs Cycle".
- Steps (List[String]): The steps themselves.

Flashcard Set

Represents a flashcard set created by a user that pertains to a specific University course.

- ID (Integer): The unique ID of the Flashcard Set.

- Name (String): Name of flashcard set
- Flashcard Set (List[Flashcard]): The flashcards in the set.
- Course (Course): The Course that the flashcard set relates to; optional.
- Creator (User): The User who created the flashcard set.
- Public (Boolean): Whether or not the flashcard set is publicly available to other users besides the Creator.
- Creation Date (Time): The date of creation of the flashcard set.
- Rating (Double): The average rating of the flashcard set, if publicly available.
- Tags (List[String]): A list of the top three most common tags in the reviews for the flashcard set, if publicly available.

University

Represents a real-life university that Users can join by providing a valid school email.

- ID (Integer): The unique ID of the University.
- Name (String): The name of the university.
- Verified Students (List[User]): A list of Users that are verified students of the University.
- Courses (List[Course]): A list of courses associated with the university.
- Email ID (String): A string used to confirm student emails.

Course

Represents a real-life course/class at a particular university. Contains data about the subject and course number of the course. Can be created by a User if it does not exist.

- ID (Integer): The unique ID of the Course.
- University (University): The University to which the course belongs to.
- Discipline (String): The official 2-4 letter subject of the course. For example, “CS” for CS 30700 at Purdue University.
- CRN (String): The identifying numbers of the course. For example, “30700” for CS 30700 at Purdue University.
- Drive (Drive): The shared drive associated with this course.
- Students (List[User]): A list of Users that are enrolled in the course.

Drive

Represents a shared drive of a particular real-life course at a university.

- ID (Integer): The unique ID of the Drive.
- Course (Course): The course associated with the shared drive.
- Students (List[User]): A list of Users that are enrolled in the university and can access the shared drive.
- Documents (List[Documents]): A list of all files within the shared drive.
- Flashcard Sets (List[Flashcard Set]): A list of all Flashcard Sets within the shared drive.

Document

Represents any non-flashcard set file in a Drive. Currently only planning to support pdfs, but more file types can be added.

- ID (Integer): The unique ID of the Document.
- Name (String): The title of the document.
- Uploader (Integer): ID of User who uploaded the document.
- Description (String): A brief description of the document under 150 characters.
- Upload Date (Time): The time and date the document was added to the drive.
- Course Date (Time): The semester and year that the document is relevant to; a required field when a User uploads the document.
- Professor (String): An optional string of a professor name that the document is relevant to.

Review

Represents a review that a user can create for a flashcard set that is not their own. Contains data about tags. The review is editable after the initial post.

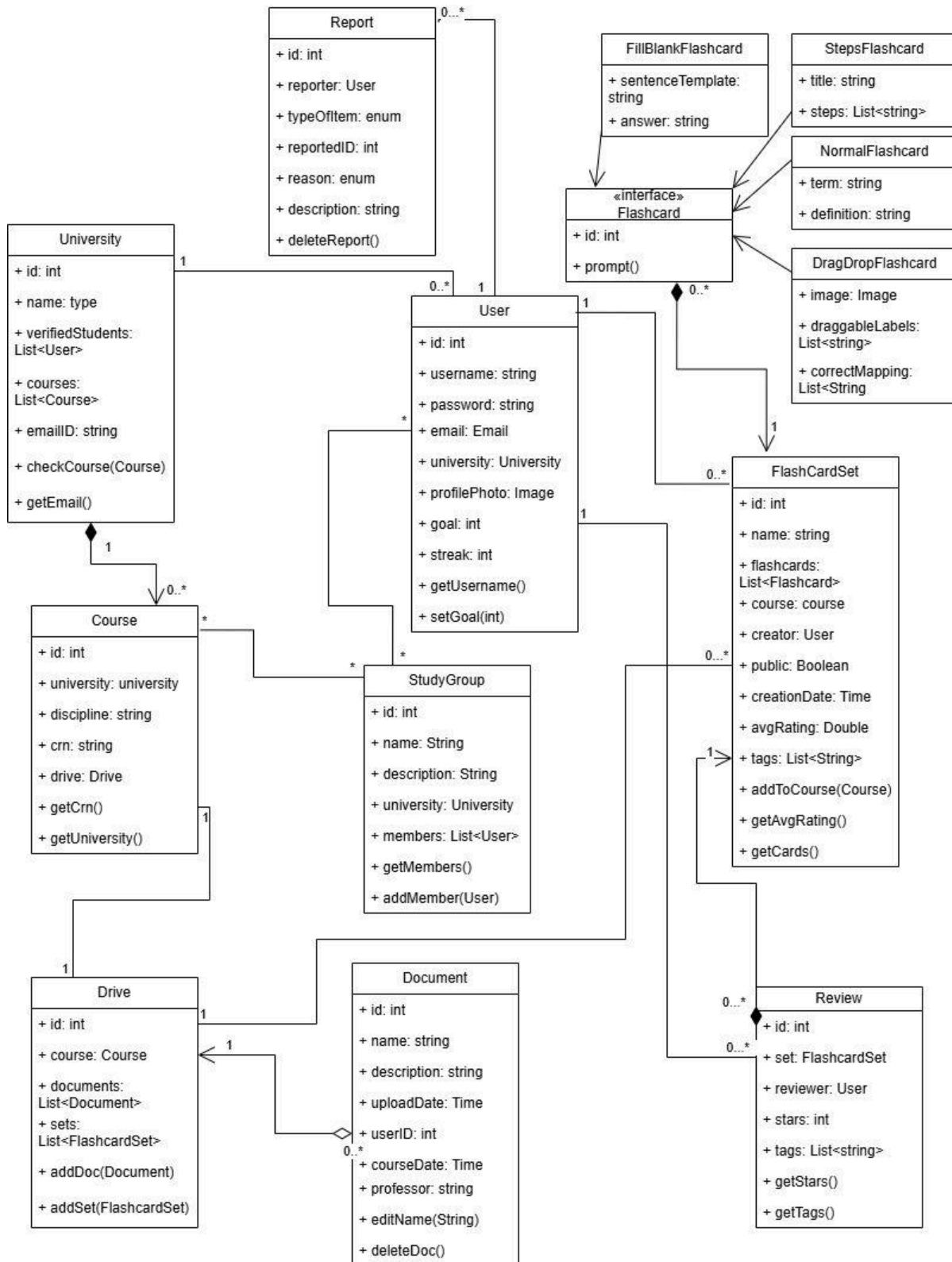
- ID (Integer): The unique ID of the Review.
- Set (Flashcard Set): The Flashcard Set that is being reviewed.
- Reviewer (User): The User leaving the review.
- Stars (Int): The number of stars associated with the review; an integer between 1 and 5.
- Tags (List[String]): Optional preset tags to describe the quality of the Flashcard Set. For example, "Easy to Study", "Similar to Classwork", "Test Questions", "Poorly Organized", etc.

Report

Represents a report that a user has created for inappropriate content on a User profile, flashcard set, shared drive materials, or other.

- ID (Integer): The unique ID of Report.
- Reporter (User): The user that submitted the report.
- Category of Item Reported (Enum): The category of the item being reported (e.g., profile, flashcard set, shared drive document, etc.)
- ID of Item Reported (Integer): The unique ID of the item being reported, fetched from the location of where the report is initiated.
- Reason (Enum): The reason for the report. Options are provided to the user when making a report. For example, “Hate Speech”, “Promoting Violence”, etc., similar to social media reports. Reports may only fall under one category.
- Description (String): An optional further description of reasoning for the report, which should go into detail about the inappropriate content in the set/user profile.

Class Interactions



Database Design

The database we picked is a Postgres SQL database so our database design has a relational philosophy built into it that ensures that the relationships between students, courses, and study materials remain consistent.

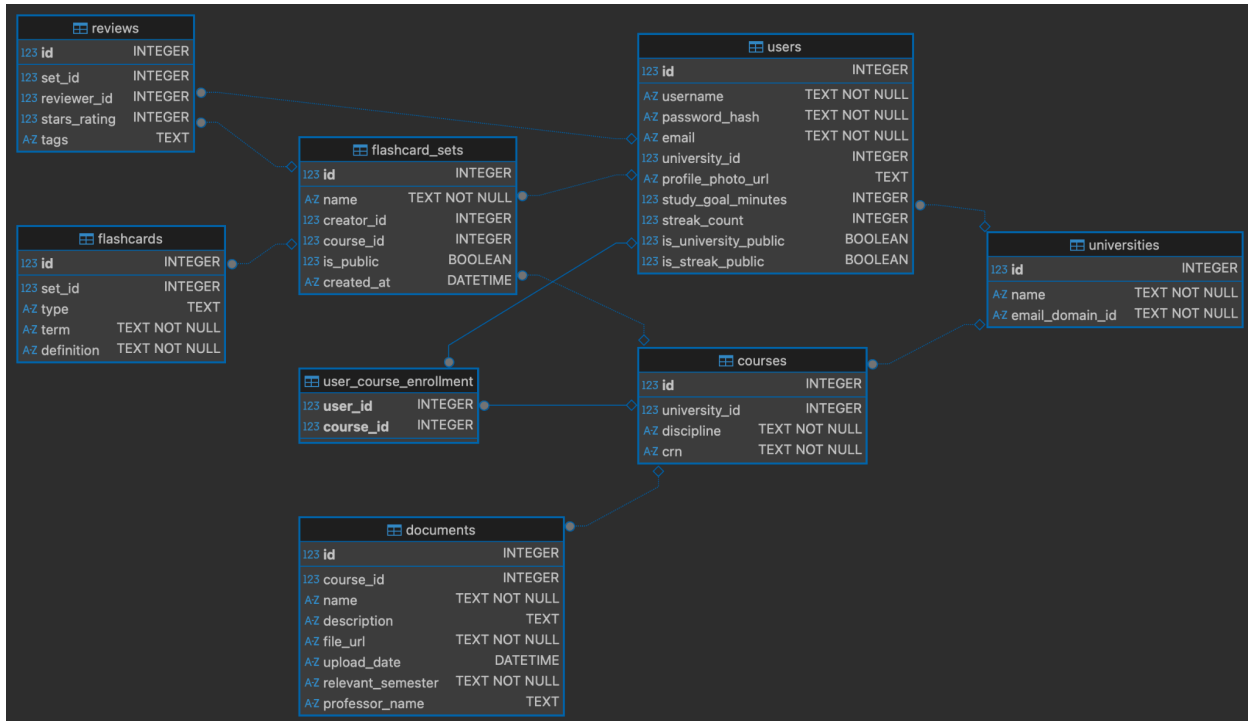
Each table in the schema directly represents a core entity within the Atama platform:

- Universities: Stores the names and email domains of supported institutions to facilitate student verification
- Users: Stores account credentials, profile details, and engagement metrics like study streaks and goals for individual students.
- Courses: Represents specific academic classes within a university, categorized by discipline and CRN.
- Flashcard_Sets: Serves as a container for individual flashcards created by users, which can be shared publicly or linked to specific courses.
- Flashcards: Contains the actual study content (terms and definitions) and supports various interaction types like "Fill in the Blank" or "Steps".
- Documents: Manages PDF-based resources in the shared drive, such as lecture notes or past exams, organized by the relevant semester.
- Reviews: Facilitates the 5-star rating and tag-based feedback system for public flashcard sets.
- User_Course_Enrollment: A join table that tracks which students are currently participating in which specific classes.

In terms of the foreign key assignment:

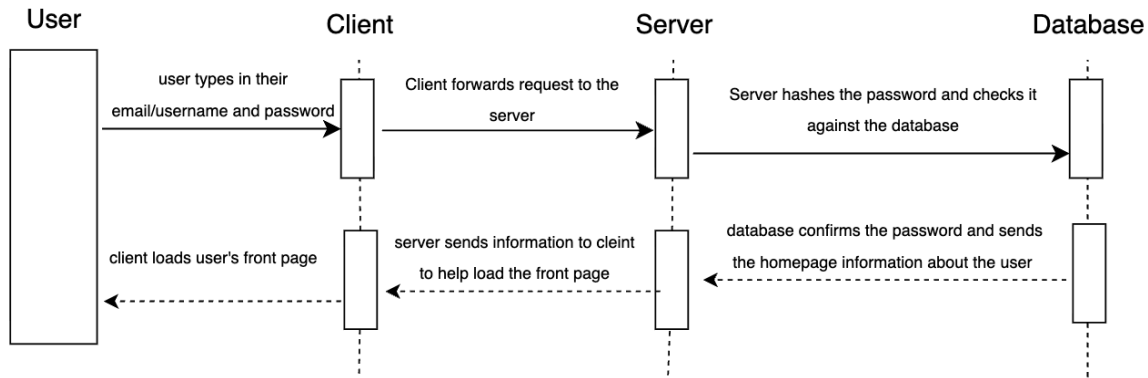
- The university_id is assigned as a foreign key in both the Users and Courses tables. This ensures that when a user searches for content, the system can automatically filter results to show materials only relevant to their specific school.
- The creator_id in the Flashcard_Sets table links directly to the Users table. This relationship is critical for managing personal libraries and allowing users to edit or delete their own materials while letting others view public sets.
- Both Flashcard_Sets and Documents use course_id as a foreign key. This architecture powers the "Shared Drive" feature, where all materials for a specific class are consolidated into one searchable location for verified students.

- Many foreign keys are set to ON DELETE CASCADE. For example, if a flashcard set is deleted, its associated individual cards and reviews are automatically removed, preventing "orphaned" data from cluttering the database and ensuring the ACID properties (consistency and durability) are maintained.



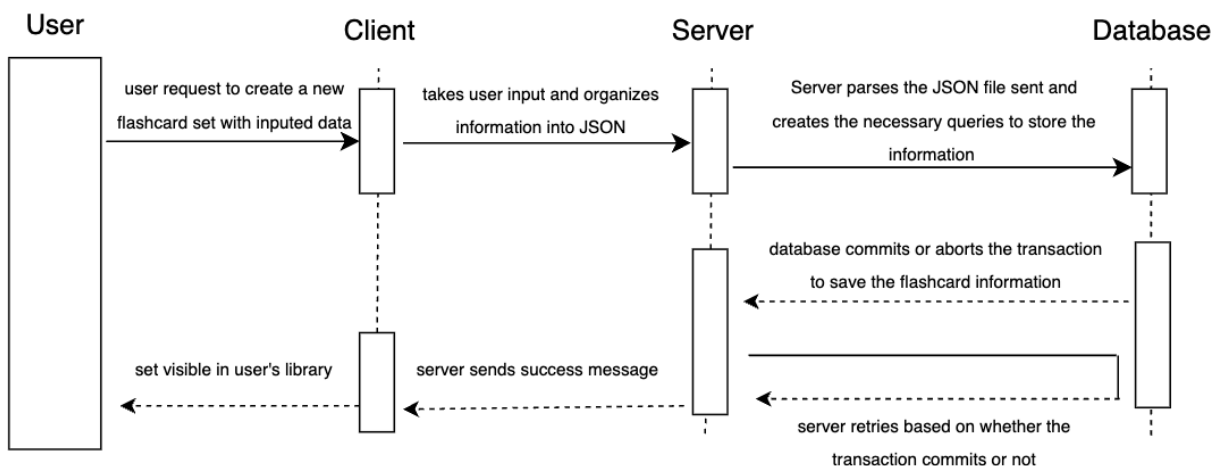
Sequence of Events Diagrams

Login and homepage



The above diagram outlines the sequence of events when a user logs into Atama and sees their homepage load. The user inputs their email and password. First the system verifies the password and sends the relevant information for their homepage. Then the client renders the homepage for the client.

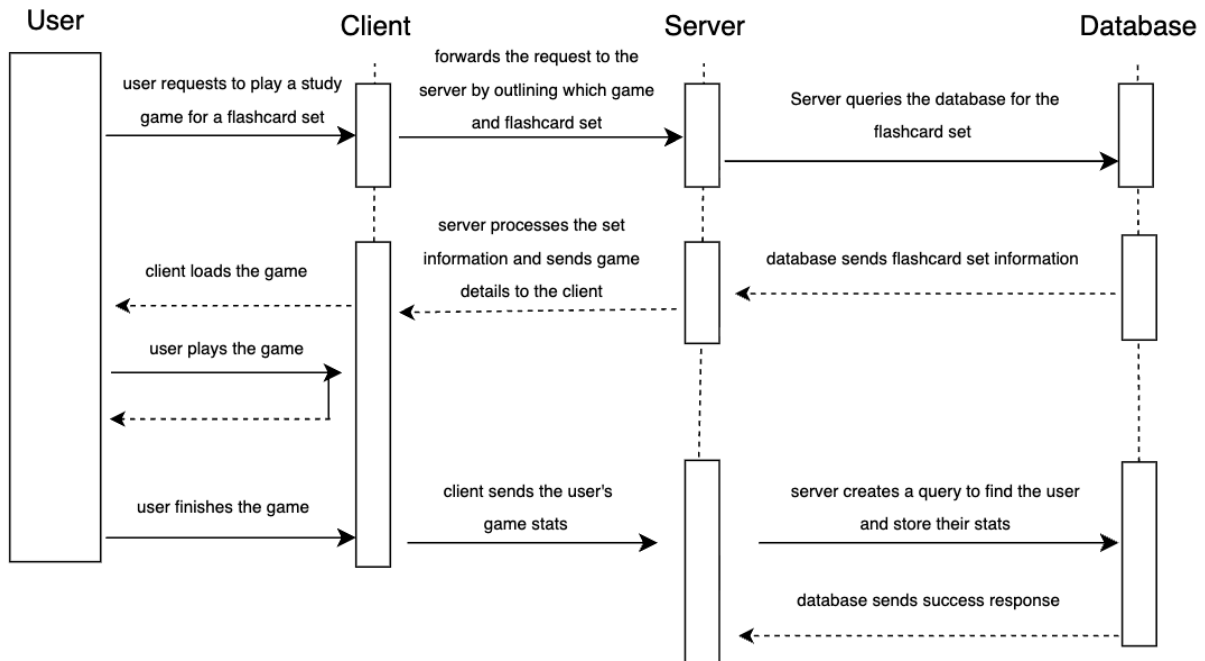
Flashcard Creation



The above diagram outlines the sequence of events when a user creates a flashcard set on Atama. The client creates a JSON file and sends it over to the server. The server creates the queries to store the flashcard set. Then it starts a transaction to commit the information. If the transaction fails, then the server retries until the information is saved.

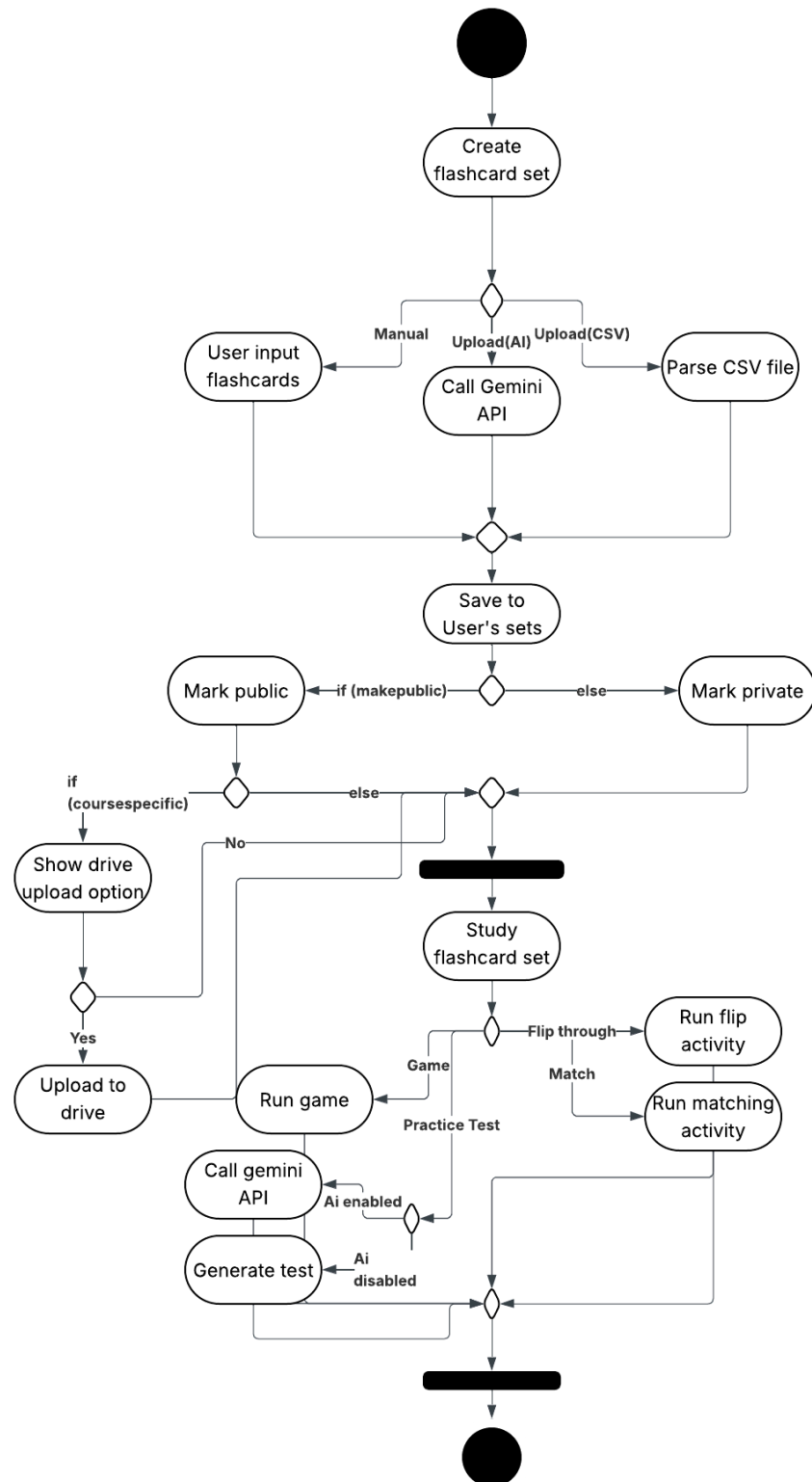
The server sends the success message to the client and the client shows that the set is in the client's library.

Playing games



The above diagram outlines the user playing a game for a flashcard set. The client sends the server the type of game and the flashcard set. The server queries the database for the flashcard set. The database sends the flashcard set to the server. The server then takes the flashcards and processes it for the game. The server then sends all of the information needed to play the game to the client. The client loads the game for the user and the user plays the game. Once the user finishes the game, the client sends the user's game stats to the server. The server saves the stats to the database.

Activity Diagram



API Routes

The API routes define the communication that happens between the client and the server. Each route represents a resource (user, flashcard, etc) that can be retrieved, created, updated, or deleted using standard HTTP methods (GET, POST, PUT, DELETE). Each resource is uniquely identified by an id, which will respond to the primary key in our SQL database.

Route	Supported HTTP Methods
/users	GET
/users/{id}	GET, PUT
/users/{id}/sets	GET, POST, PUT, DELETE
/users/{id}/sets/{setId}	GET, POST, PUT, DELETE
/users/{id}/sets/{setId}/flashcards	GET
/users/{id}/sets/{setId}/flashcards/{cardId}	GET, POST, PUT, DELETE
/users/{id}/stats	GET
/universities	GET
/universities/{id}	GET, POST
/universities/{id}/courses	GET
/universities/{id}/courses/{courseId}	GET, POST, DELETE
/ai/generateFlashcards	POST
/ai/generateTest	POST